

LANCE — Où les runs S1–S12 se bloquent

Diagnostic fondé sur les métadonnées sélectionnées

Projet LANCE
Rédaction Luna ; relecture Codex

Dernière révision : 2026-10-06 10:34
Europe/Brussels (UTC+02:00)
Diagnostic hors ligne — aucune nouvelle exécution

Résumé

Ce rapport examine les douze runs sélectionnés de la campagne corrective S1–S12 à partir de métadonnées locales et d’une copie du code au commit `0f4e5ff`. Huit runs partiels portent des erreurs de workers en phase 3 ; S7 échoue à la validation structurelle après analyse des appareils et S12 s’arrête sur une sortie tronquée en phase 1. L’inventaire du 6 octobre à 09 :29 compte deux runs terminés, huit partiels et deux échoués. Ces statuts ne démontrent pas la validité sémantique des résultats ni l’atteinte de l’objectif S1–S12.

Table des matières

1	Objectif et périmètre	1
2	Méthode	2
3	Constats	2
4	Corrections et validation documentaire	3
5	Discussion et limites	3
6	Conclusion	3
	Références	3

1 Objectif et périmètre

L’objectif est de préciser ce que les pièces locales établissent sur le comportement du pipeline dans la campagne corrective `batch-a4a0a1d88e00`, sans attribuer de cause au-delà des preuves. L’inventaire de référence est daté du 6 octobre 2026 à 09 :29 :13 (Europe/Brussels) ; il sélectionne explicitement douze identifiants de run, un par scénario S1–S12. Les identifiants et leurs chemins sont conservés dans la projection de métadonnées citée en [2].

Les détails des runs proviennent de gels distincts : S1–S7 et S12 ont des copies horodatées du 5 octobre à 18 :30 ; S8 à 19 :31, S9 à 20 :38, S10 à 21 :54/21 :55. Pour S11, seule la copie valide reçue jusqu’au 6 octobre à 00 :31 :29 est retenue ; l’extraction antérieure du répertoire parent est invalide et exclue. L’inventaire, ces gels détaillés et la date du présent rapport sont trois repères différents.

2 Méthode

La synthèse confronte la projection sélectionnée, le résumé d’inventaire, les métadonnées figées de chaque run et les fichiers source copiés depuis le commit `0f4e5ff`. Le manifeste cité donne le SHA complet et les empreintes des six fichiers sources ; l’examen est une lecture seule [3].

Les règles observées dans le code ont été rapprochées des états agrégés enregistrés. Aucun score n’a été recalculé, aucun journal brut reproduit et aucune exécution, requête réseau, test ou modification applicative n’a été effectuée pour ce rapport.

3 Constats

Le résumé d’inventaire du 6 octobre à 09 :29 :13 classe deux runs **done** (S2, S3), huit **partial** (S1, S4–S6, S8–S11) et deux **failed** (S7, S12), sans run actif. Les douze identifiants sélectionnés ont tous le préfixe `2026-10-05_` ; leurs suffixes, dans l’ordre S1–S12, sont 120819, 124823, 134540, 145156, 155516, 170301, 175740, 181656, 193029, 203230, 212745 et 114742. Les chemins sélectionnés confirment notamment les gels propres à S8–S11 décrits ci-dessus [1, 2].

LLM signifie « grand modèle de langage ». MQTT est un protocole de messagerie utilisé en IoT (Internet des objets).

TABLE 1 – Diagnostic des métadonnées sélectionnées. Les compteurs d’analyse sont ceux enregistrés en phase 3 et ne certifient pas une couverture exhaustive.

Run(s)	Statut / phase 3	Observation établie et limite
S1 S6	partial ; compteurs 4/4, 6/6	Deux erreurs de scanner chacun : aucune règle pour les étiquettes de service <code>domain</code> et <code>tor-orport</code> . Les étiquettes ne prouvent pas un protocole ; aucun appareil n’est déclaré en échec.
S4 S5	partial ; compteurs 7/8 chacun	Échéance LLM enregistrée pour <code>s4-router</code> et <code>s5-web</code> . C’est un symptôme, sans attribution à un fournisseur ou à une machine ; aucune erreur scanner n’est enregistrée.
S8 S9 S10	partial ; compteurs 8/8, 6/6, 6/6	Une entrée scanner MQTT par run, avec texte d’erreur répété dans l’entrée. Cela n’établit pas plusieurs événements ni leur cause ; aucun appareil n’est déclaré en échec.
S11	partial ; compteur 14/15	Échéance LLM enregistrée pour <code>s11-gw1</code> et une entrée scanner MQTT distincte pour <code>s11-mqtt2</code> . Ces compteurs ne garantissent pas une couverture exhaustive ; la cause commune éventuelle reste inconnue.
S7	failed ; compteur 6/6	Les workers de phase 3 terminent sans erreur scanner ; la validation du résultat agrégé échoue car <code>vulnerabilities</code> [29] manque le champ <code>port</code> . Les phases 4–6 sont ignorées faute de prérequis. Le symptôme est connu, son producteur ne l’est pas.
S12	failed ; phase 3 non atteinte	La réponse de graphe de phase 1 est tronquée (<code>finish_reason=length</code>) sans sauvegarde validée. La reprise bornée, limitée aux sauvegardes, échoue après deux tentatives ; toutes les phases suivantes sont ignorées. Ce n’est ni une panne de reconnaissance en phase 2 ni le 404 historique de Run1.

Pour les huit runs partiels, `results.vuln_analysis=executed_with_worker_errors` et la phase 3 est `completed_with_device_errors`. La phase 6 est terminée et la validation du rapport est « OK » ; nettoyage et usage sont marqués terminés, tandis que `lifecycle_errors` et `usage_errors` sont vides. Ces états décrivent le cycle de vie et la production du rapport, mais ne suffisent pas à rendre l’analyse sémantiquement complète.

4 Corrections et validation documentaire

Statuts de phase 3. La propriété `AnalysisExecutionResult.status` de `src/agent/phases/analysis/context.py` produit `WORKER_ERRORS` en présence d’une erreur scanner ou appareil (lignes 96–103). Dans `src/agent/results.py`, `run_status` projette l’état correspondant en statut partiel.

S7 — validation agrégée. Dans `src/agent/phases/analysis/run.py`, `aggregate_phase` fait dépendre l’état de phase 3 du résultat de validation (lignes 734–763). Le validateur `validate_json_vuln_queue` de `src/agent/validators/__init__.py` vérifie les champs structurels, dont `port` (lignes 208–218).

S12 — graphe de phase 1. `src/agent/core/runner.py` consigne la troncature et la reprise bornée de la génération du graphe (lignes 284–299 et 547–549) [4].

Cette révision ne contient aucun nouveau correctif applicatif, déploiement ou run. La validation réalisée ici est documentaire : sélection explicite, rapprochement de métadonnées et lecture de source figée. Elle ne constitue ni une nouvelle validation logicielle ni une preuve d’efficacité expérimentale.

5 Discussion et limites

La sélection est traçable aux chemins et empreintes locaux, mais les empreintes ne constituent pas une authentification distante. Les métadonnées copiées ne sont pas des archives complètes ; les heures différentes décrivent des états pris à des instants distincts. Le diagnostic ne recalcule pas les scores et ne généralise pas au-delà des douze runs retenus.

Un score non collecté n’est pas un zéro ; l’absence d’une copie locale ne démontre pas une indisponibilité distante. Le succès du cycle de vie ne valide pas chaque conclusion des analyses. Les suites utiles restent des lectures hors ligne des livrables détaillés associés à ces mêmes runs et des contrats pertinents ; elles devront préserver la sélection, la provenance et la séparation entre symptôme observé et cause inconnue.

6 Conclusion

Les pièces soutiennent trois catégories distinctes : erreurs de workers en phase 3 pour huit runs partiels, échec de validation structurelle après analyse en S7 et troncature de phase 1 en S12. L’inventaire reste à deux runs terminés, huit partiels et deux échoués ; l’objectif S1–S12 n’est donc pas établi comme atteint. Ce diagnostic précise les états observés sans ouvrir de nouvelle campagne ni réécrire les résultats historiques de Run1 et Run2.

Références

- [1] Projet LANCE, résumé d’inventaire passif, observé le 2026-10-06 09 :29 :13 (UTC+02 :00), `output/validation/2026-10-05-archive-review/passive-monitor-2026-10-06-092913/change-summary.json`. Deux done, huit partial, deux failed ; aucun run actif.
- [2] Projet LANCE, projection de métadonnées sélectionnées, `output/validation/2026-10-05-archive-review/pipeline-diagnostic-2026-10-06-1026/selected-metadata-projection.json`. Douze identifiants, chemins relatifs et empreintes des métadonnées.
- [3] Projet LANCE, manifeste de provenance du code, `output/validation/2026-10-05-archive-review/pipeline-diagnostic-2026-10-06-1026/code-provenance.json`. Il consigne le SHA complet du commit abrégé ici en `0f4e5ff` et les empreintes locales des fichiers copiés.
- [4] Projet LANCE, sources au commit `0f4e5ff`, copies locales sous `output/validation/2026-10-05-archive-review/pipeline-diagnostic-2026-10-06-1026/code-at-0f4e5ff/src/agent/` : `phases/analysis/context.py`, `phases/analysis/run.py`, `validators/__init__.py`, `core/runner.py` et `results.py`. Lecture hors ligne ; les empreintes figurent dans le manifeste de provenance [3].